

A2

Changze Wu

1008837840

March 2024

1 Q1

Proof:

Let $r = \text{size}(\mathbf{R})$, $s = \text{size}(\mathbf{S})$, $t = \text{size}(\mathbf{T})$ at the time of the rotation.
 v is left-heavy, so either

- a node was added to x to cause imbalance, or
- a node was removed from $\mathbf{R}$ to cause imbalance.

Assumption:

- $\text{weight}(v.\text{left}) > \text{weight}(v.\text{right}) * 3 \Rightarrow 3(r+1) < s + t + 2$
- $3(r+1) < s + t + 2$

Case 1: Before addition, we had a WBT:

- $r+1 \leq 3(s+t+1)$ and $s+t+1 \leq 3(r+1)$
- $t \leq 3(s+1)$ and $s \leq 3(t+1)$

From the above, we conclude below:

- $3r+1 \leq s+t \leq 3r+2$
- $s < 2t+1$
- $r \leq 3s+3t+2 \leq 12t+11$
- $t \leq 3s+3$
- $s \leq 3t+3$

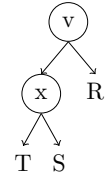


Figure 1: Diagram

Since s, t, r are all integer, we get $s+t = 3r+2$. $\frac{1}{3}(t+1) \leq s + \frac{4}{3} \leq s+2+r$, for $r \geq 0$

Since $3r+1 \leq s+t \Rightarrow r < \frac{s+t-1}{3} \Rightarrow s+2+r < \frac{4s+t+5}{3} < \frac{4(2t+1)+t+5}{3} = 3t+3$, for $s < 2t+1$

Hold $\frac{1}{3} \leq \frac{t+1}{s+r+2} \leq 3$

Since $r < \frac{s+t-1}{3}$ and $t \leq 3s+3 \Rightarrow r+1 < \frac{s+3s+3-1}{3}+1 = \frac{4s+2}{3}+1 < 3s+3$, for $s \geq 0$

Since $s+1 \leq s+t+1 \leq 3r+3$, for $t \geq 0$

Hold $\frac{1}{3} \leq \frac{r+1}{s+1} \leq 3$

Case 2: Before removal, we had a WBT:

- $r+2 \leq 3(s+t+2)$ and $s+t+2 \leq 3(r+2)$
- $s+1 \leq 3(t+1)$ and $t+1 \leq 3(s+1)$

From the above, we conclude below:

- $3r+1 \leq s+t \leq 3r+4$
- $s < 2t+1$
- $r \leq 3s+3t+4$
- $s \leq 3t+2$
- $t \leq 3s+2$

Since $\frac{1}{3}(t+1) \leq s+1 \leq s+2+r$, for $r \geq 0$, $t \leq 3s+2$

Since $3r+1 \leq s+t \Rightarrow r < \frac{s+t-1}{3} \Rightarrow s+2+r < \frac{4s+t+5}{3} < \frac{4(2t+1)+t+5}{3} = 3t+3$, for $s < 2t+1$

Hold $\frac{1}{3} \leq \frac{t+1}{s+r+2} \leq 3$

Since $r < \frac{s+t-1}{3}$ and $t \leq 3s+2 \Rightarrow r+1 < \frac{s+3s+2-1}{3}+1 = \frac{4s+1}{3}+1 < 3s+3$, for $s \geq 0$

Since $3r+1 < s+t < 3t+1 \Rightarrow r \leq t-1$, and it is removal, for $r \geq 1$, $s+1 \leq s+t-1 \leq 3r+3$

Hold $\frac{1}{3} \leq \frac{r+1}{s+1} \leq 3$

Both case hold, QED.

2 Q2

2.1

Define two struct typeof named **node** and **func**.

node: contains two int value named *key* and *size*(the function rebalance will use it but i haven't given it in my below code) and two node pointer value *left* and *right* to indicate left and right child.

func: contains two node pointer value named *left* and *right* to indicate $T_{<k}$ and $T_{>k}$, and also have one boolean value *b* to indicate if the key in origin WBT.

And there are also to function named ***newFunc(node left, node right, bool b)** and ***newNode(int key)** each parameter corresponding in the definition

There are also a function named **balance(node)** to re-balance the tree

There

Algorithm 1 struct func split(node *T,int k)

```
1: struct func func
2: if T == nil then
3:   func→left = NULL
4:   func→right = NULL
5:   b = false
6:   return result
7: end if
8: if k < T→key then
9:   struct func leftsplit = split(T→left, k)
10:  func→left = leftsplit→left
11:  func→right = join(leftsplit→right, T→right)
12:  func→b = leftsplit→b
13:  balance(func→left)
14:  balance(func→right)
15:  return func
16: else if k > T→key then
17:  struct func rightsplit = split(T→right, k)
18:  func→left = join(T→left, rightsplit→left)
19:  func→right = rightsplit→right
20:  func→b = rightsplit→b
21:  balance(func→left)
22:  balance(func→right)
23:  return func
24: else
25:  func→left = T→left
26:  func→right = T→right
27:  func→b = true
28:  balance(func→left)
29:  balance(func→right)
30:  return func;
31: end if
```

2.2

Aside the **node** type we create before. Currently create a new **pair** which contained two *node* pointer variable named *tree* and *max*

Algorithm 2 struct Pair remove_max(node* T)

```
1: struct pair result;
2: if T == NULL then
3:   result→tree = NULL
4:   result→max = NULL
5:   return result;
6: end if
7: if T→right == NULL then
8:   result→tree = T→left;
9:   result→max = T;
10:  return result
11: end if
12: struct pair rightsub = remove_max(T→right);
13: T→right = rightsub→tree;
14: result→tree = balance(T);
15: result→max = rightsub→max;
16: return result;
```

2.3

Algorithm 3 struct node *join(struct node* L, struct node* G)

```
1: if L == NULL then
2:   return G
3: end if
4: if G == NULL then
5:   return L
6: end if
7: struct pair result = remove_Max(L)
8: struct Node* L_prime = result→tree;
9: Node* max = result→max
10: struct Node* T = join(L_prime, G)
11: T = insert(T, max)
12: return T
```

Algorithm 4 struct Node* insert(struct node* root, struct node* new)

```
1: if root == NULL then
2:   return new
3: end if
4: if new == NULL then
5:   return root
6: end if
7: if new→key < root→key then
8:   root→left = insert(root→left, new);
9: else
10:  root→right = insert(root→right, new)
11: end if
12: root = balance(root)
13: return root
```

Algorithm 5 void balance(node* T)

```
1: if weight(v.right) > weight(v.left) * 3: then
2:   let x = v.right
3:   if weight(x.left) > weight(x.right) * 2: then
4:     single rotation: counter-clockwise
5:   else
6:     double rotation: clockwise then counter-clockwise
7:   end if
8: else if weight(v.left) > weight(v.right) * 3: then
9:   let x = v.left
10:  if weight(x.right) > weight(x.left) * 2: then
11:    single rotation: clockwise
12:  else
13:    double rotation: counter-clockwise then clockwise
14:  end if
15: else
16:   no rotation
17: end if
```

Algorithm 6 struct node* intersection(struct node *T1, struct node *T2)

```
1: struct node *T = malloc(sizeof(struct node))
2: T = NULL
3: if T1 == NULL — T2 == NULL then
4:   return NULL;
5: end if
6: int array[1024]
7: createArray(T1, &array, 0)
8: int i = 0;
9: while isdigit(array[i]) do
10:  T = insert(T, search(T2, array[i]))
11:  i++
12: end while
13: return T
```

Algorithm 7 void createArray(struct node *T, int *array, int index)

```
1: while T != NULL do
2:   array[index] = T.key
3:   createArray(T.left, &array, index+1)
4:   createArray(T.right, &array, index+1)
5: end while
```

Algorithm 8 node * search(struct node *T, int key)

```
1: if root == NULL then  
2:   return NULL;  
3: end if  
4: if T.key == key then  
5:   return T;  
6: end if  
7: if T.key > key then  
8:   return search(T.left, key);  
9: else  
10:  return search(T.right, key);  
11: end if
```

3 Q4

Adjacency-list for left graph	Adjacency-list for right graph	Adjacency-list for wrong graph
1 → 2 → 4 → 3	1 → 4 → 3 → 2	1 → 2 → 3 → 4
2 → 5 → 6	4 → 6 → 7	2 → 5 → 6
4 → 7	3 → 5	3 → 5 → 7
3	2	4 → 6 → 7
5	6	5
6	7	6
7	5	7

According to third adjacency list, whenever first pass 2 or 3 or 4, the first should have two child, it is impossible for each one have one child.

4 Q5

4.1

visit	enqueue
a	→ b
b	→ a
c	→ g → b
d	→ c
e	→ f → a
f	→ c → a
g	→ h → f
h	→ d

4.2

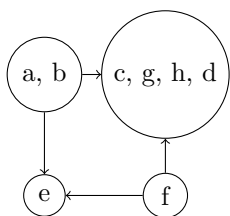
	a	b	c	d	h	g	f	e
discover	1	2	3	4	5	6	10	11
finish	16	15	14	9	8	7	13	12

Table 1: $a \rightarrow b \rightarrow c \rightarrow f \rightarrow e \rightarrow d \rightarrow h \rightarrow g$

4.3

$\{a, b\}$ $\{c, g, h, d\}$ $\{e\}$ $\{f\}$

4.4



4.5

First, we need to establish that the strongly connected components of G and G^T are the same. By definition, two vertices u and v belong to the same strongly connected component if there exists a path from u to v and a path from v to u . This property remains unchanged when we transpose

the graph since transposing only reverses the direction of edges, but it doesn't affect the existence of paths between vertices.

The component graph represents the strongly connected components of a graph as vertices, with edges indicating connectivity between these components. Since the strongly connected components remain the same for G and G^T , the component graphs of both G and G^T will have the same set of vertices (representing SCCs). The edges in the component graph are determined by the edges between SCCs in the original graph. Transposing a graph doesn't change the connectivity between SCCs, as it only reverses edge directions.

When we transpose G^T , we essentially reverse the direction of edges in G^T , but this doesn't affect the structure of strongly connected components or their connectivity. Therefore, the component graph of $(G^T)^{SCC}$ remains the same as the component graph of G .

Thus, we've shown that $((G^T)^{SCC})^T = G^{SCC}$, demonstrating that the transpose of the component graph of G^T is indeed the same as the component graph of G .

4.6

I think it works, the reasons below:

The first DFS on the transpose of the graph (G^T) identifies the finish times of vertices. This step remains unchanged in the simplified algorithm. In the simplified algorithm, the second DFS is performed on the original graph G . The crucial point is that the order of visiting vertices in the second DFS does not affect the identification of SCCs. SCCs are determined by the structure of the graph, not the order in which vertices are visited. Visiting vertices in order of increasing finish times will still correctly identify SCCs because it ensures that we explore vertices within the same SCC before moving to vertices in other SCCs. Since the identification of SCCs is unaffected by the order of visiting vertices in the second DFS, Anya and Albert's simplified algorithm will correctly find SCCs in the graph G . Therefore, the simplified algorithm proposed by Anya and Albert is valid and will correctly identify the SCCs of the graph G .